ChatGPT

基于事件驱动的MT5自动跟单系统技术文档

1. 系统概述与需求

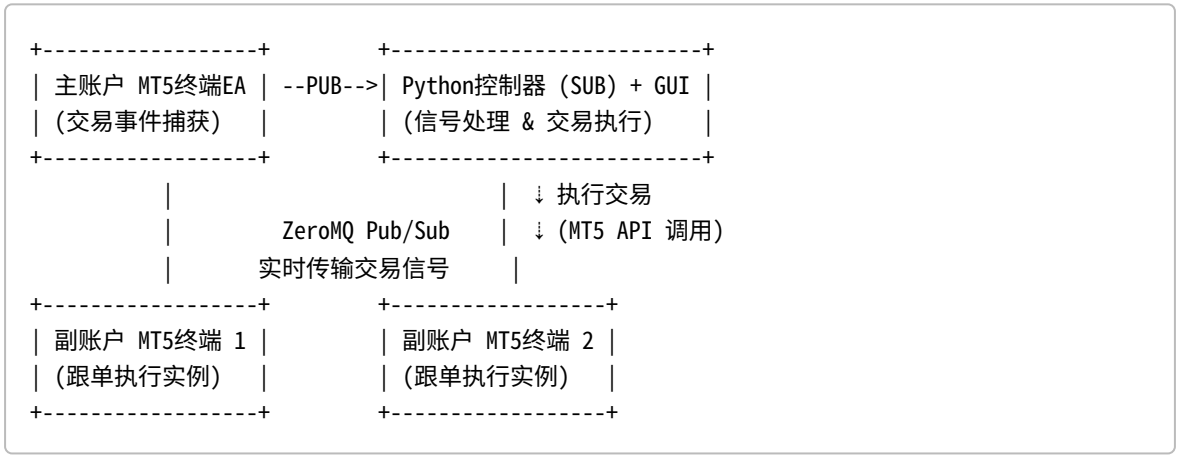
本技术文档提出一种优化的MT5多账户自动跟单系统方案，整合多份参考方案的优点，旨在实现**低延迟、高可扩展性**的自动跟单功能。所有MT5终端（主账户和两个副账户）与Python控制脚本运行在**同一台机器**上，每个副账户使用独立的MT5实例 [1] [2]。系统通过在主账户运行EA（Expert Advisor）捕获交易事件，并使用ZeroMQ消息队列将信号推送给Python控制器，实现事件驱动的跟单（**避免API轮询**） [3]。Python作为主要开发语言构建核心逻辑，保证模块化设计和高扩展性，并结合图形用户界面（GUI）方便用户操作和监控。本方案满足以下具体需求：

- **本地同机运行**：所有组件部署在同一物理机，最大程度减少网络延迟 [4]。副账户采用独立MT5终端实例运行（可通过复制MT5安装目录实现），如有更优多账户接入方案会在下文讨论 [2]。
- **事件驱动信号捕获**：采用主账户端EA的`OnTradeTransaction`事件监听交易 [5]，通过ZeroMQ **实时推送**信号，替代Python API的轮询扫描 [3]。这样可在毫秒级捕获主账户操作，并将交易同步到副账户。
- **Python主控，模块化设计**：以Python为主要开发语言，利用其官方MT5接口和丰富生态，实现核心跟单逻辑 [6]。系统按功能划分模块，支持后续横向扩展（增加账户数量、接入新策略）或纵向扩展（远程部署、多平台支持） [7]。
- **图形界面**：提供友好的GUI，支持**启动/暂停**跟单、策略参数设置（正向/反向跟单切换等）、止盈止损设置、手数调整、账户添加管理，以及交易监控等功能 [8] [9]。
- **高效配置管理**：账户凭据、策略参数等配置数据采用简洁高效的存储方式。考虑到数据规模较小且读写频率有限，推荐使用**JSON配置文件**存储，便于直接编辑和维护 [10]。如未来需要管理大量数据，可考虑引入轻量级数据库（如SQLite）以提高并发读写效率。
- **示例场景**：当前系统配置3个账户（1主2副），其中副账户1执行正向跟单、副账户2执行反向跟单。系统支持多品种交易，包含市价单与挂单的同步复制，以及止盈止损的镜像设置，以确保两种跟单账户在行情变化时盈亏表现相反但幅度对称 [11]。

2. 系统架构设计

2.1 模块架构与通信

本跟单系统采用**主从分离**的三层架构，包括：**主账户事件捕获层**、**核心控制器层**、**从账户执行层**，并辅以用户交互的GUI组件 [12] [13]。各模块通过ZeroMQ消息队列解耦通信，实现低延迟和高可靠性。架构图如下：



通信流程：主账户MT5上的EA将交易事件封装成消息，通过ZeroMQ **发布/订阅**模式发送；Python控制器作为订阅者实时接收消息并解析交易指令 [14] [15]。控制器根据每个副账户的配置（正向或反向跟单、手数倍率等）处理信号，然后通过MT5 Python API将对应的交易指令下达到各副账户终端 [16] [15]。这种解耦架构保证主账户信号能**一对多**地广播到多个跟随端，并行执行下单，最大程度降低延迟。

2.2 核心组件描述

系统核心由以下模块构成：

- **主账户EA模块（信号发布者）**：运行于主账户MT5终端的MQL5脚本（EA）。其职责是监听一切交易相关事件，利用`OnTradeTransaction`事件处理函数捕获订单下达、成交、修改、平仓等事务 [5]。EA将交易详情（包括品种、订单类型、方向、手数、价格、止损止盈等）序列化为JSON字符串，然后通过ZeroMQ **PUB**套接字将消息发布出去 [17] [18]。这个EA只专注于事件捕获和发送，不包含跟单决策逻辑，以确保在交易发生的源头以**极小开销**快速发送信号 [19]。利用MQL5内置事件机制可实现**零延迟**捕获交易，为毫秒级同步奠定基础 [20]。

- **Python控制器模块（信号订阅者）**：这是系统的大脑，由Python脚本实现。它通过ZeroMQ **SUB**套接字订阅主账户EA发布的交易信号 [21]。收到消息后，控制器将JSON解析回交易指令对象，并执行**业务逻辑**：包括根据副账户配置决定是否复制该交易、以正向还是反向方向下单，以及调整交易手数等 [16]。控制器与多个副账户MT5终端建立连接，通常每个副账户对应一个独立连接会话。由于MetaTrader5官方API在单一Python进程中默认只能维持一个终端连接 [2]，控制器可采用**多进程或多线程**架构：主进程负责接收信号和调度，各副账户执行由单独的线程或子进程负责，以并行执行交易指令 [22]。控制器模块保证对每个信号的处理和分发在**毫秒级**完成，避免因串行处理多个账户而造成延误 [23]。该模块对外提供接口（例如函数或消息）供GUI调用，以便实现启动/停止跟单、添加删除账户、调整策略参数等操作。

- **副账户执行模块（交易执行者）**：每个副账户对应一个独立的执行单元，通常在Python控制器中新建一个连接实例或子进程与该MT5账户交互 [22]。执行模块接收来自控制器的标准化交易指令，然后通过MetaTrader5 Python API在对应副账户上下单、修改订单或平仓 [24]。它需要处理各种订单操作的结果：如下单成功则记录订单ID并反馈给控制器，若失败则返回错误信息便于后续处理。执行模块还应维护副账户的当前仓位和订单映射关系，以便在主账户触发平仓或挂单变化时能定位并操作对应的副账户订单 [25]。各执行模块相互独立，彼此通过控制器协调，这种**主从解耦**设计方便扩展额外副账户，而无需改变主控制逻辑 [7]。

- **账户管理与配置模块**：负责存储和提供系统运行所需的账户信息及跟单策略参数。包括主账户和所有副账户的登录凭据（账号、密码、服务器地址），以及副账户的个性化设置：如跟单方向（正向/反向）、手数调整方式（倍率或固定手数）、是否复制挂单、是否镜像止盈止损等策略选项 [26] [27]。这些配置数据保存在本地JSON文件中以简化实现（文件示例见下文），Python控制器在启动时读取配置并建立相应的账户连接 [28] [29]。与JSON文件交互简单直接，能满足小规模配置管理需求 [10]；如未来需求增加，可无缝切换到SQLite等嵌入式数据库以获得更高并发和数据完整性保障。出于安全考虑，建议对本地存储的敏感信息（如账户密码）进行**加密或混淆**处理 [10]。在运行中，账户管理模块应提供接口函数，支持GUI添加新账户或修改现有配置，通过更新JSON文件或内存数据结构并通知控制器动态加载变化。

- **图形用户界面模块（GUI）**：为用户提供启动/停止系统和实时监控的交互窗口。采用PyQt5框架构建桌面应用GUI [30]（也可考虑Tkinter或其他框架；若需跨设备访问可扩展为Web界面，但桌面GUI更快捷可靠 [31]）。GUI主要包括以下界面元素 [9]：

- **账户管理区**：列出现有主账户和副账户列表及其关键设置（账号ID、服务器、余额等），并提供按钮或表单以添加新账户或编辑/删除现有账户 [32]。可通过下拉框或切换开关选择每个副账户的跟单模式（正

向或反向）[26]，输入框设置手数倍数或固定手数[27]。对于每个账户，可显示其连接状态（在线/离线）[33]。

- **策略控制区**：包含跟单策略的全局控制，如**开始跟单**与**暂停跟单**的按钮（全局开关），以及针对每个副账户的启用/停用复选框。当用户点击“暂停账户B跟单”时，系统应能暂时停止将交易信号发送到该账户的执行模块（例如控制器忽略该账户的信号分发），而不影响其他账户[34]。此外还有参数设置项，如是否复制挂单、是否同步止盈止损的选项开关，以防用户根据不同平台情况做调整[35]。
- **实时监控区**：显示主账户与各副账户的当前交易状态和日志信息[36][37]。包括：主账户当前持仓列表及每笔持仓对应的副账户跟单情况（比如副账户订单号、开仓价、浮动盈亏等）；系统交易日志（列出每条信号及其在各账户执行结果，如下单成功/失败及时间戳）；系统状态信息（如最近信号处理延迟、错误警告、心跳状态）等。日志区域可以是滚动的文本框，记录重要事件和错误，供用户排查问题[38]。为了不影响界面响应，日志和状态更新通过PyQt的**信号/槽**机制从后台线程异步推送到界面上[39]。

GUI模块与后台控制器逻辑松散耦合：GUI一般运行在主线程，而交易复制逻辑可能在后台线程或异步任务中。通过使用线程安全的通信机制（如PyQt的signals/slots、共享队列或ZMQREQ/REP套接字等）传递指令和状态，可避免直接跨线程操作界面引发的不稳定[40]。例如，当用户点击“开始”按钮时，GUI触发信号调用控制器的启动函数，控制器开始订阅信号和执行交易；当交易信号到来时，控制器处理后将更新结果通过信号发给GUI，GUI再刷新交易列表和日志。这样的设计保证了GUI即使在刷新大量数据时，核心跟单引擎仍在独立运行，不受阻塞[41]。

2.3 配置文件示例

下面给出一个JSON配置文件片段示例，展示如何存储多账户信息和策略设置：

```
{
  "master": {
    "login": "100001",
    "password": "master_pass",
    "server": "Broker-Demo",
    "mt5_path": "C:\\Program Files\\MetaTrader5\\terminal64.exe"
  },
  "followers": [
    {
      "name": "Follower1",
      "login": "200001",
      "password": "follower1_pass",
      "server": "Broker-Demo",
      "mt5_path": "C:\\MT5Instances\\MT5_1\\terminal64.exe",
      "type": "regular",
      "lot_mode": "multiplier",
      "lot_multiplier": 1.0,
      "copy_pending": true,
      "copy_sl_tp": true
    },
    {
      "name": "Follower2",
      "login": "200002",
      "password": "follower2_pass",
      "server": "Broker-Demo",
      "mt5_path": "C:\\MT5Instances\\MT5_2\\terminal64.exe",
```

```
        "type": "reverse",
        "lot_mode": "multiplier",
        "lot_multiplier": 1.0,
        "copy_pending": true,
        "copy_sl_tp": true
      }
    ]
}
```

上述配置表示主账户和两个副账户（Follower1正向跟单，Follower2反向跟单），每个副账户包含独立的MT5终端路径、登录凭据、跟单模式及策略选项等 [42] [43]。在实际实现中，可扩展此配置结构以支持更多账户或更多策略参数。系统启动时，Python控制器将读取该配置并尝试登录各账户；GUI启动后，会根据配置填充界面元素，并允许用户修改部分参数（如调整 `lot_multiplier` 或切换 `copy_pending` 开关），修改可即时保存回配置文件或通过控制器接口应用。

3. 跟单策略逻辑详解

跟单策略逻辑是系统的核心，决定主账户每个交易如何复制到副账户，以及在不同情形下采取何种操作。系统需支持**正向跟单**和**反向跟单**两种模式，并处理**市价单**和**挂单**等各种订单类型的同步，确保止损止盈等风险控制措施在跟单过程中有效镜像。

3.1 正向跟单与反向跟单

正向跟单：副账户执行与主账户**相同方向**的交易，复制完全相同的订单类型和参数。例如主账户开多仓（Buy），副账户也跟随开多仓；主账户设置止损价和止盈价，副账户同步设置相同数值的止损/止盈。这是默认的跟单模式。

反向跟单：副账户执行与主账户**相反方向**的交易，以对冲主账户的头寸。具体而言，主账户的Buy单将被副账户复制为Sell单，Sell单复制为Buy单 [26]。对于挂单，反向模式需要将订单类型一并反转：例如主账户的Buy Limit挂单（低价买入）在副账户应下达Sell Stop（低价突破卖出） [26]；主账户的Buy Stop挂单（高价买入突破）应对应副账户的Sell　Limit（高价卖出挂单）。如此保证当主账户订单触发时，副账户获得相反方向的仓位。反向跟单对止损止盈的处理也有所不同：**交换SL/TP**的位置，即主账户多单的止损价将作为副账户空单的止盈价，主账户多单的止盈价则作为副账户空单的止损价 [11]。通过这种镜像设置，确保对于同一行情波动，正向跟单账户和反向跟单账户的风险/收益是对称的 [11]（例如主账户触发止损亏损时，反向账户的止盈获利也被触发）。

在实现上，每个副账户都可以独立设置为正向或反向模式。控制器在接收到交易信号后，会查看目标副账户的模式： - 若为正向模式（`type="regular"`），则按信号所示方向下单； - 若为反向模式（`type="reverse"`），则对信号做**方向翻转**后下单，并对止盈止损参数做交换处理 [11] [26]。　该逻辑适用于市价单和平仓操作，以及各种挂单类型。在附带示例代码中，会体现这一判断和处理。

3.2 市价单跟单

市价单（即时成交单）的复制相对直接。当主账户下达市价单（Buy或Sell）时，EA会在 `OnTradeTransaction` 中捕获该订单成交事件并发送交易细节，例如：交易品种（符号）、方向、手数、主账户成交价格等 [44]。Python控制器接收后，立即针对每个已启用跟单的副账户执行下单操作：　-　对于正向跟单账户，提交**相同品种、相同手数、相同方向**的市价单。 - 对于反向跟单账户，提交**相同品种、相同手数、相反方向**的市价单。

由于不同账户可能在不同券商或服务器，市场报价可能略有差异，因此副账户市价下单可能出现**滑点**或被拒绝的情况（例如价格变动超出允许范围） [45]。为提高成功率，系统应采取以下措施 [45]： - **容错价差**：允许一定

价差范围。如果副账户下市价单因“价格偏移”被拒，控制器可以在误差容限内重新尝试一次下单，或调整为以当前市价重新下单 [45]。 - **异步并行执行**：主控制器不必等待所有副账户下单完成再返回。应**并行**触发各副账户的下单请求（可通过多线程或`asyncio.gather`并发执行），并分别记录每个结果 [45]。这样一个副账户的延迟或失败不会阻塞其他副账户的执行，提高整体速度。 - **结果处理**：对于每个下单结果，若失败则记录错误日志（包含时间、原因、订单详情）并提示GUI [46]。可根据失败原因决定是否进一步重试。例如由于临时网络故障导致未下单，可立即再次尝试；若因资金不足或品种不可交易导致失败，则不再重试但需要通知用户人工干预 [47]。

通过上述策略，系统在主账户市价单出现后的几毫秒内即可在多个副账户尝试复制仓位，努力确保各账户仓位保持一致。

3.3 挂单跟单

挂单（止损单和限价单）的复制和同步相对复杂。主账户下达挂单时（Buy Limit、Sell Limit、Buy Stop、Sell Stop等），EA将捕获挂单创建事件并发送信号。控制器处理时，应在每个副账户**以对应的挂单类型**下单 [48]：正向跟单则下相同类型挂单，反向跟单则下反转类型的挂单（如前述Buy Limit→Sell Stop） [26]。

挂单同步的难点在于后续状态变化： - **挂单触发**：当主账户挂单被市场价格触发执行（转变为持仓）时，控制器应收到主账户的新持仓信号以及挂单状态变化。此时需要检查副账户对应的挂单是否也已成交。如果**副账户挂单也成交**，则双方仓位同步；如果**副账户挂单尚未成交**（可能因为副账户报价未触及触发价），系统需决定如何处理。为保持仓位同步，一种策略是在检测到主账户挂单已成交后，立即以市价强制在副账户建仓 [49]（即使挂单价格未到，也不再等待），确保副账户跟上主账户的仓位变化节奏。另一种策略是允许副账户挂单继续等待自己的触发条件，但这将导致主副账户的建仓时点可能不一致，从而出现风险暴露差异 [49]。综合考虑，本方案倾向于**严格同步**：主账户挂单一旦成交，若副账户对应挂单未成交则转为市价下单进场 [49]；主账户挂单被手动取消或过期取消时，立即撤销所有副账户对应挂单 [49]。这样可以最大程度保证所有账户的持仓状态一致。 - **挂单修改**：如果主账户修改了未触发挂单的参数（价格、手数或止损止盈），EA应发送交易修改事件。控制器接收后需要同步修改副账户上对应挂单的参数。可通过`mt5.order_modify()`等API函数调整价格或止损等。如副账户订单ID不同步（比如主账户挂单曾重新下单），需要通过映射关系准确找到副账户挂单进行修改。 - **不复制挂单选项**：在特殊情况下，用户可能选择不复制主账户的挂单，仅在主账户订单实际成交后再让副账户以市价跟进。正如某些EA提供的选项，“跟挂单”如果关闭则只跟进最终的成交单 [35]。本方案默认复制挂单以求同步，但如果所连接的平台报价差异较大、挂单同步困难，可以提供参数关闭挂单复制。这种模式下，系统会忽略主账户的挂单创建信号，仅在主账户实际建仓时再在副账户直接市价建仓。

3.4 平仓同步

当主账户**平仓**某笔持仓（无论是手动平仓还是止盈/止损自动触发），主账户EA会捕获订单关闭的事件。控制器接收该信号后，应立即在所有副账户中寻找并**平掉对应仓位** [25]。为实现这一点，控制器需在下单时维护一张**订单映射表**（或持仓映射）：将主账户每笔订单的Ticket号或Magic Number关联到各副账户对应订单的Ticket [25]。当主账户发出关闭信号时，根据映射迅速定位各副账户持仓并执行市价平仓操作。若副账户存在部分仓位（如因为复制失败导致没有该仓位），则记录异常并通知用户 [50]。

在反向跟单模式下，主账户平仓有时意味着主账户可能止损出场（亏损平仓），此时反向账户的对应持仓通常处于盈利状态。然而，为保持动作一致，**反向账户也应同步平仓** [51]——除非策略另有规定允许独立持仓管理。这样可确保账户之间不出现**敞口不匹配**的情况，也避免一个账户已经无仓位而另一个账户还持仓的风险。

同步平仓的及时性至关重要。如果因为通信延迟导致副账户平仓滞后，可能引发额外风险。因此系统在实现上应将平仓信号视为**最高优先级**事件处理，一旦检测到主账户平仓，立即中断或跳过队列中其他非紧急任务，先行处理平仓指令。

3.5 止损止盈同步

止损（SL）和止盈（TP）的设置同步是跟单系统风控的重要部分。当主账户修改持仓的止损或止盈水平时，应将相同的调整应用到副账户对应持仓上 [52]。对于**正向跟单**，此时只需将新的SL/TP价格直接设置到副账户仓位。对于**反向跟单**，如前所述，需要**互换SL和TP**后再应用：例如主账户多单将止损上调到某价格，意味着副账户空单应当将止盈上调到该价格 [11]。

需要注意的是，如果副账户的手数与主账户不同（例如根据资金比例调整手数），则尽管SL/TP价位相同，但对账户资金的实际影响比例会不同。不过一般仍建议**按主账户原价同步**止损止盈 [52]，以忠实复制主账户的交易策略。否则，如果尝试按资金比例调整止损距离，可能改变策略的风险收益特性。

此外，在某些跨平台情形下，复制止损止盈也可能导致副账户提前或延后触发。例如两账户报价略有差别，副账户可能比主账户稍早达到止盈价而平仓。这方面需要在策略上权衡：一种做法是允许这种小的时差；另一种是提供参数允许用户选择“不复制止损止盈”，由用户手动或另行风控（某些EA就提供该选项以应对不同平台价格差 [53]）。总的来说，为完全复制交易策略，本方案默认同步止损止盈，并假设账户间价差不至于严重影响风控触发。如果用户担心此问题，可关闭该选项并接受风险暴露差异。

3.6 手数调整和风控

每个副账户可以根据自身资金规模或用户偏好，选择不同的手数跟单策略： - **固定手数**：无论主账户下单多少手，副账户都按预设的固定手数下单 [27]。这种模式简单粗暴，但适合资金量相近的账户保持固定风险。 - **倍率跟单**：按主账户手数乘以某倍率进行下单 [27]。倍率可以是1（完全相同手数），也可以小于1（减小仓位以降低风险）或大于1（放大仓位以提高收益，但也提高风险）。倍率还可根据账户余额比例动态设定，例如副账户余额是主账户的50%，则倍率设为0.5以用相同比例资金交易。 - **智能比例**（可选）：更复杂的策略可按账户净值比例、保证金占用比例等动态计算手数，使得各账户风险敞口更匹配。但这超出本方案范围，需谨慎设计。

此外，为了增强风控，系统可实现如下机制： - **延迟/价格偏差过滤**：可配置跳过某些滞后订单。例如如果主账户订单已成交超过N秒才检测到（因为通信延迟或系统刚启动时发现已有旧持仓），可选择不再复制该笔交易，以免追单风险 [54]。同样可设定价格偏差阈值：如果当前市场价格相对于主账户成交价已经朝不利方向超过X点，则放弃复制这笔单 [55]。这些参数可在配置中设置，避免跟进明显不利的情况。 - **Magic Number和备注**：在MT平台上下单时可以设置订单的Magic Number和注释。可以利用Magic区分跟单订单与手工订单，或不同副账户来源的订单。这有助于风险隔离和调试 [56]。Magic Number还可用于高级用途，如配合其他EA识别特定单子 [56]。 - **声音或弹窗提醒**：在跟单成功或失败时提供提示（声音/消息） [57]以提示用户关注。例如下单被拒绝时立即弹出警告，提醒用户检查账户状况。 - **日志记录与异常报警**：系统应详尽记录每次信号处理的结果，以及发生的错误（包括时间、涉及账户、错误原因） [47]。GUI上可以直观显示或导出日志，并在出现连续错误或长时间未收到心跳（见下节）时，通过UI高亮、声音等方式报警 [58]。完善的日志和报警有助于**安全性**，可及时发现问题（如某副账户掉线、执行失败）并加以处理。

4. 数据流说明

本章节描述一次交易从主账户触发到副账户执行的完整数据流，展示系统内部信号和数据的流动过程：

1. **交易触发**（主账户）： 主账户的交易终端上发生一个交易事件。例如交易者手动下达了一笔EURUSD的Buy 1.0手市价多单，或原有持仓Hit了止盈被平仓，抑或EA修改了一笔订单的止损价。此事件在主账户MT5中触发`OnTradeTransaction`回调 [5]，主账户EA模块捕获到该事件对象，其中包含交易类型（下单/平仓/修改等）以及涉及的订单和仓位详情。

2. **序列化发送**（主账户EA）： EA将事件中的关键信息提取并组装成**消息对象**，然后序列化为JSON字符串 [17]。例如，对于开仓事件，JSON可能包含：`{"action": "open", "symbol": "EURUSD", "type": "BUY", "volume": 1.0, "price": 1.1000, "sl": 1.0900, "tp": 1.1200, "ticket": 12345}` 等字

段。EA通过ZeroMQ的PUB套接字将该JSON消息发送到预设的地址（如`tcp://*:5555`） [18]。由于采用发布/订阅模式，EA不需要等待确认即可迅速返回执行，其发送是异步非阻塞的 [59]。

3. **消息传输**（ZeroMQ网络）： ZeroMQ库在后台负责将消息从PUB端高效传输给所有已连接的SUB客户端 [60]。在本方案中，Python控制器作为订阅者运行在同一主机，通过`tcp://localhost:5555`订阅，所以消息经本地环回接口传输，延迟极低（理论上几十微秒级）。ZeroMQ自动处理了底层套接字的连接、队列和重连等细节，保证消息可靠送达 [59]。如果GUI和控制器分离为不同进程，GUI也可以作为另一个SUB订阅同一主题获取实时交易流（本方案中GUI直接通过共享对象获取数据，因此不额外订阅）。

4. **信号接收**（Python控制器）：Python控制器的ZeroMQ SUB套接字监听端口`5555`上的所有主题（或约定的主题）消息 [21]。当主账户EA发布消息时，控制器立刻接收该消息字符串。通过Python的`json.loads`解析出原始交易信号数据结构。

5. **信号处理**（Python控制器）：控制器根据消息内容的`action`和订单信息，决定采取何种操作：

6. 如果`action`表明是**新订单开仓**（如上例Buy开仓），则触发“下单”流程：为每个**已启用**跟单的副账户生成对应的下单请求（考虑各账户正反向和手数倍率）并加入执行队列。
7. 如果是**订单关闭/平仓**事件，则触发“同步平仓”流程：查找对应的副账户订单并加入平仓队列。
8. 如果是**挂单创建**事件，则在副账户创建对应挂单或决定忽略，加入下单队列。
9. 如果是**订单修改**（如修改止损），则加入一个“修改订单”任务。
10. …对于不同类型事件均有相应处理分支。

控制器将这些任务异步分发给副账户执行模块。可以为每个副账户维护一个独立任务队列或采用线程池并发执行，下文代码结构部分将对此有示例。此阶段还会根据副账户配置过滤不需要执行的情况，如某账户处于暂停状态，则跳过给该账户派发任务。

1. **指令执行**（副账户执行模块）：对于每个需要执行的任务，调用MetaTrader5 Python API对相应MT5终端发送命令。例如：
2. 对应开仓：构造`order_request`字典并调用`mt5.order_send(request)`执行市价下单或下挂单。
3. 对应平仓：找到持仓position并调用`mt5.Close(position.ticket)`或再发送反向市价单平仓（MT5 API支持直接平仓函数）。
4. 对应修改止损：调用`mt5.order_modify(order_ticket, new_sl, new_tp, ...)`等。执行模块会阻塞等待每次API调用返回结果（通常几十毫秒以内完成）。拿到结果后，根据返回的`retcode`判断成功或失败：
5. 成功：如果是下单成功，会得到新订单的ticket，将其与主订单ticket一起记录到映射表中 [25]。若是平仓成功，则更新映射表移除该订单关联。如果是挂单创建成功，也要记录映射关系等。

6. 失败：记录错误日志，包括失败原因`mt5.last_error()`，如果有重试策略则执行重试（例如价格问题可以调整价格重下）。重试若仍失败则放弃，并标记该副账户该信号为失败，在GUI上将状态标红提示 [47]。

7. **结果回馈与状态更新**：执行模块将结果通知主控制器主线程，或者由控制器主动检查任务执行结果。控制器随即：

8. 更新内部仓位和订单追踪数据结构。如开仓成功则增加副账户仓位记录，平仓成功则移除记录。
9. 通过GUI刷新接口或事件，将该笔信号的执行状态更新到界面。例如在交易日志表中添加一行“时间X: 主账户Buy 1.0手EURUSD -> Follower1执行成功（order #4567），Follower2执行成功（order #8910）”。
10. 若有失败，则在日志中标记失败原因，如“Follower2执行失败：交易品种不可用”并通知用户检查配置。

11. 保持对**延迟**的监测：记录从主账户信号时间到各副账户完成的时间差，用于评估性能。如发现某账户延迟过大，可在GUI上高亮提示（可能因为该账户网络或硬件性能问题，需要优化） [61] 。

12. **后续调整**：当主账户后续对仓位进行止盈止损修改、部分平仓等操作时，会再次触发上述流程中的相关步骤，直至该交易完全闭环结束。整个过程中，系统始终保持事件驱动、实时响应的运行方式，没有采用轮询主账户状态，从而极大降低了延迟和资源消耗 [3] 。

通过以上数据流，可以看到每一笔交易信号都经历了发布、传输、接收、处理、执行和反馈的链路。由于采用高效的ZeroMQ消息队列和本地API调用，链路上的每一步都进行了优化：在主账户端**即时推送**，在通信层**毫秒传输**，在控制端**并行处理**，在执行端**快速下单**，确保跟单的整体延迟被压缩在**亚秒级**范围内甚至数十毫秒之内。这满足了高频交易环境对跟单系统的严格要求。

5. 技术选型与实现方案

本节对关键技术选型进行说明，并给出实现层面的建议。

5.1 开发语言与API

Python被选为主要实现语言，原因在于： - MetaTrader官方提供了Python API库，使Python程序可直接与本地MT5终端交互 [6] 。通过该API，Python能执行登录、查询行情、交易指令发送等操作，且与MT5终端通过本地DLL通信，时延极低 [62] 。 - Python有丰富的金融计算和交易相关库，可用于扩展功能，例如数据分析、风险控制算法等 [63] 。其强大的科学计算生态对构建交易策略十分有利。 - Python易于编写多线程或异步代码，通过 `asyncio` 或线程池可实现高并发任务处理 [64] 。虽然有GIL限制，但在I/O密集（等待网络/磁盘/MT5响应）时，异步编程可提高吞吐。 - Python拥有成熟的GUI框架如PyQt5、Tkinter等，可以较容易地集成图形界面 [65] 。相比之下，**JavaScript**虽然在Web应用上很强，但缺乏直接的MT5集成且需要借助Electron等方案实现桌面GUI，复杂性更高 [66] 。Node.js通过第三方云API也能跟单，但引入外部依赖和潜在延迟，不如本地Python方案直接 [67] 。 - 综合比较，Python方案**开发效率高**且**延迟低**，更适合本地多账户跟单 [68] 。

因此，本系统选择**Python 3.x**作为开发语言，使用**MetaTrader5 Python API**连接MT5终端 [69] ，并结合**ZeroMQ**和**PyQt5**构建通信和界面 [29] 。

5.2 通信机制

ZeroMQ作为核心通信中间件，提供主账户EA与Python控制器之间高效的消息通道。选用ZeroMQ主要基于以下考虑： - ZeroMQ是嵌入式的消息库，不需要单独部署消息服务器，直接集成进EA和Python脚本，大大降低复杂度和延迟 [59] 。 - 提供多种通信模式，尤其适合本场景的发布/订阅(PUB/SUB)模式，可轻松实现一对多广播 [70] 。主账户EA无需关心有多少副账户在监听，也不必为每个副账户单独发送，降低了发信负担。 - ZeroMQ对**异步I/O**和**多线程**支持良好。EA端可在后台异步发送消息，Python端可在独立线程收取，不阻塞各自主要流程。 - 在本地通信时，ZeroMQ性能极高。对比传统TCP Socket需要处理底层细节，ZeroMQ自动完成连接管理、消息队列和重连等功能，代码简洁且健壮 [59] 。 - 可扩展性：未来如果需要分布式部署（主账户EA和控制器不在同一机器），ZeroMQ的通信也适用，并可通过Curve或其它机制加密通信确保安全。

另外，对于GUI和控制器之间的交互，也可以选择使用ZeroMQ的**请求/应答(REQ/REP)**模式或**PAIR**套接字，以实现跨进程或分布式的界面控制。如果GUI与控制器在同一进程，则可直接调用函数或使用Qt信号，暂不需要额外通信框架。

替代方案：HTTP或WebSocket也可用于主账户与控制器通信，但HTTP每次建立连接开销大且需要Web服务支持，不够实时；WebSocket在低延迟和二进制处理上不及ZeroMQ简单高效，且没有现成MQL5绑定库。因此ZeroMQ是此处的最佳选择 [59] 。

5.3 多账户连接策略

正如前文提及，MetaTrader5官方Python库默认**同一进程**只能连接一个终端（一个账户）。为控制多个副账户，有几种策略： 1. **多实例多进程**：在本地安装多个MT5终端（可通过复制MT5安装目录创建独立实例），主机上每运行一个MT5实例登录一个账户 [2] 。Python控制器使用**多进程**或`multiprocessing`模块，每个子进程负责一个账户的API连接。主进程通过进程间通信（队列/管道等）向子进程发送交易指令，子进程执行后回传结果。这一方案隔离性好，各账户互不干扰，可充分利用多核CPU并行下单，是推荐方案 [22] 。实现稍复杂但较稳健。 2. **多线程+库复制**（非常规方案）：有开发者通过复制`MetaTrader5` Python模块文件为多个不同名字（如MetaTrader5_1，MetaTrader5_2），然后在同一进程中导入多份模块，从而维持多连接 [22] 。或者尝试在一次`initialize()`后注销再初始化另一个账户。这些技巧非常不官方，可能存在不稳定因素，不做推荐。 3. **轮询切换**（低效，不采用）：理论上可以在一个线程内循环切换登录不同账户执行操作，但这完全无法并行且有大量重登录开销，行不通。

综合考虑，**多进程多实例**是目前控制多个MT5账户的主流方案 [22] 。对于当前仅2个副账户的规模，也可以简单地在Python中起两个线程分别调用各自MT5终端（通过环境变量或短时锁定调用），但随着账户增加将难以维持。所以本系统架构按多进程设计扩展性更好。在实现中，可让每个副账户模块fork新进程并调用`mt5.initialize(path=对应终端)`登录账号，然后等待来自主控制器的指令。

5.4 GUI框架选择

GUI部分本方案选用了**PyQt5**，其优势在于： - 提供丰富的**GUI组件**和设计工具(Qt Designer)，能快速搭建复杂界面 [8] 。例如表格、树控件可用于账户和持仓列表，图形图表可用于盈亏曲线展示等（如需扩展）。 - **跨平台**支持良好，在Windows/Linux上体验一致。 - **信号/槽**机制让多线程交互变得安全便利 [30] 。尤其当控制器在后台子线程运行时，可用Qt的signals在线程间传递信息，避免直接操作界面。 - 有大量PyQt和交易系统结合的实践，可以借鉴现有组件如图表控件、日志高亮等。

替代框架如Tkinter虽然简单但界面美观和复杂布局方面不足；Kivy适合跨平台移动端但不如PyQt在桌面端成熟；Web界面通过Flask/Dash构建虽然跨平台但实时性和本地集成度稍弱 [31] 。因此PyQt是较务实的选择。

在PyQt实现时，需要注意将核心交易逻辑与UI解耦，以免阻塞GUI主线程 [34] 。建议使用**QThread**将控制器运行在后台线程，并通过信号通知GUI更新界面 [41] 。测试表明，对于几十个账户以内的场景，使用QThread线程切换开销可以忽略不计，而且与PyQt集成成熟 [71] [72] 。只有在上百个并发连接的超大规模场景下，才需要考虑使用`asyncio`异步模型并配合适当的GUI刷新机制 [71] 。目前需求3个账户，用多线程完全能胜任且实现简单。

5.5 存储和配置管理

配置数据存储选型上，**JSON文件**足以满足需求 [10] 。JSON优点是直观、可人读编辑，同时被Python内置库原生支持解析读写。鉴于配置文件主要在启动加载和用户调整后写入，访问频率低，使用文件不会成为瓶颈。

YAML也是可考虑的格式，支持写注释，结构更灵活，但需引入PyYAML库。相比JSON的普及度和简单性，还是倾向JSON即可。

SQLite等嵌入式数据库适合在需要频繁查询、更新大量配置记录时使用。本系统配置项有限，读写频率也不高，引入数据库反而增加复杂度。除非未来需要扩展支持上百个账户配置，或记录交易历史、统计数据等，这时可将这些信息存入SQLite以方便查询。

配置管理需确保**敏感信息安全**。一方面，JSON文件本身可通过合理的文件权限保护不被任意用户读取/修改；另一方面，考虑使用简易的加密，如将密码字段用简单的对称加密（例如Fernet或Windows DPAPI）保存，而在运行时由控制器解密加载 [10] 。虽然本地运行的系统风险较低，但基本的防护仍有意义。

5.6 关键性能优化

要实现毫秒级响应，需在以下方面优化： - **事件推送替代轮询**：正如方案采用的，在MT5端利用事件驱动推送取代Python定时轮询查询，大幅减少延迟和CPU浪费 [3] 。轮询频率即便设为50ms也不如事件瞬时触发，本方案的EA+ZMQ机制是性能基石 [73] 。 - **本地通信**：所有组件（MT5、Python、GUI）尽量在同机甚至同进程内通信，省去网络传输延迟 [74] 。例如控制器和副账户终端在本地调用DLL交互，速度远高于跨网络API。 - **并行处理**：充分利用多线程/多进程，将信号处理、多个账户下单并行化，避免串行瓶颈 [23] 。特别对于同一时刻主账户批量下多单的情况，并行对副账户下单非常必要。 - **异步I/O**：如使用 `asyncio` ，可将ZMQ接收、日志写入等I/O操作异步化，不阻塞主逻辑 [74] 。不过实际测试中，用线程也可胜任且代码简单。 - **序列化优化**：默认使用JSON序列化便于调试，但为了极致性能可考虑MessagePack或Protocol Buffers来替换，减少消息大小和解析时间 [75] 。在当前3账户场景下尚无必要，但保留为优化方向。 - **算法优化**：确保跟单逻辑中的计算高效，如避免重复遍历列表、尽量使用字典索引映射订单等。对于可能产生大量事件的场景，需要定期profiling性能 [76] 。 - **资源复用**：MT5连接初始化等耗时操作只做一次，之后重复利用连接。避免频繁打开/关闭连接或读取硬盘数据。比如EA可以保持ZMQ socket长连接而不是每次重建。 - **批量操作**（可选）：MT5 API没有直接批量下多单接口，但可以并行调用达到类似效果。若将来有支持，也可利用。

5.7 可靠性与安全

除性能外，系统需注重可靠性和安全性： - **异常捕获**：对所有和MT5 API、网络通信相关的操作都应有异常捕获 [77] 。任何 `mt5.order_send` 或ZMQ发送失败都不应导致程序崩溃，而是记录错误并尝试局部恢复。 - **日志和审计**：保留关键操作日志，最好将日志写入文件保存，以备审计和问题追溯 [47] 。可考虑实现日志自动滚动和定期清理，防止文件无限增大。 - **心跳检测**：可让主账户EA每隔几秒发送一条心跳消息（特殊主题），GUI和控制器监测心跳间隔 [58] 。如果一段时间未收到主账户心跳，界面应警告用户EA或连接可能异常。这有助于及时发现主账户掉线或EA停止工作等情况。 - **数据完整性**：考虑网络抖动可能造成ZMQ消息丢失（虽概率极低）。在关键交易信号下发时，可以引入**同步确认**机制：例如控制器在成功收到交易后通过REP回应EA一次，EA可以在下一次事件中检查上次信号是否有确认，否则可重发 [78] 。这属于高级优化，基本PUB/SUB已可靠的情况下可根据需要实现。 - **访问控制**：如果系统运行在开放网络环境中，ZeroMQ端口可能被他人连接滥用。因此需要设置防火墙限制或使用ZeroMQ自带的Curve加密认证机制确保只有授权进程可以发送/订阅消息。 - **账户安全**：上文提到的凭据加密存储、防止日志泄露密码等都属于账户安全措施。可以进一步把账户管理整合Hardware Security Module或操作系统凭据管理，以达到更高安全级别（视需求而定）。

6. 可扩展性与后续改进

本系统采用的模块化架构，使其具备相当的可扩展性。各组件之间通过明确定义的接口通信，新增功能时可以局部修改或添加模块而不影响整体。

- **账户扩展**：支持增加更多副账户跟单。得益于发布/订阅模式，无论副账户数量变多少，主账户EA发送信号逻辑无需变化 [79] 。控制器可动态创建新的执行模块实例处理新账户即可 [80] 。GUI也可扩展列表显示更多账户。实践中几十个账户在一台机器上也是可行的，但需注意MT5终端和硬件资源上限。如果需要上百账户，可以将副账户分布到多台服务器运行，各服务器运行一个控制子进程订阅同样的主账户信号，这也是架构可扩展的一种体现。
- **跨平台/远程扩展**：虽然目前假定所有终端在本地，但通过ZeroMQ网络传输，**完全可以将副账户终端部署在异地服务器**上，只要网络足够可靠低延迟（如VPS靠近交易服务器） [13] [81] 。可以在远程服务器上运行Python控制子进程连接该服务器的MT5，再由主控制器通过ZMQ推送信号给它。类似地，GUI也可以远程连接控制器获取数据，比如做一个Web版界面。这样的扩展允许主账户和跟单账户不在同一机器，便于全球布署来尽量贴近各交易中心，减少交易执行延迟。
- **功能扩展**：模块化设计便于添加新功能而不破坏现有组件 [7] 。例如，可增加**风险管理模块**：监控整体仓位风险，当达到预定阈值时发送警报甚至自动平仓所有账户。又或者增加**策略模块**：允许对跟单信号做过滤（如只跟特定品种，或主账户某EA的订单等，通过Magic区分）。由于主控制器将EA信号标准化为数据结构，这些模块都可作为控制器的插件对数据进行处理。

- **策略多样化**：除了正向/反向，还可扩展**条件跟单**（满足某条件才跟）或**差异化跟单**（不同副账户按不同规则跟不同的主账户策略）。通过引入策略对象来封装跟单规则，每个副账户绑定一个策略实例，实现更灵活的配置。
- **性能升级**：可根据需要替换更高性能组件而不影响整体。如消息通信层改用共享内存或更底层的FPGA NIC加速，在极端延迟场景下使用 [82] 。又如将关键逻辑用C扩展实现，以获得更高执行速度。由于当前结构清晰，这些优化都可以逐步实现。
- **高可用**：未来可考虑引入**冗余主账户**或**灾备**机制。例如允许同时监听两个主账户信号源，当一个失联时自动切换，以避免单点故障。这需要EA发布端的配合（可能通过不同主题区分不同主账户）和控制器的协调。

总之，本方案从一开始就注重分层解耦和接口标准化，使得系统能够从一个专业级脚本逐步成长为庞大的交易网络，而无需推翻重来 [7] 。独立的组件还便于调试和测试，如可以对控制器编写单元测试（模拟输入信号看输出操作），对GUI进行独立测试等，提高维护性。

7. 样例代码结构与模块封装

下面给出系统实现的代码结构建议（伪代码和模块划分），以供开发时参考：

```
project_root/
├── main.py                # 程序入口，初始化GUI和控制器
├── config.json            # 配置文件（账户信息和策略参数）
├── ea/
│   └── TradePublisher.mq5 # 主账户EA源码（MQL5），发布交易事件
├── controller/
│   ├── __init__.py
│   ├── controller.py      # 控制器主模块：订阅ZMQ、处理信号调度
│   ├── trade_executor.py  # 交易执行类：连接MT5并执行订单
│   ├── account_manager.py # 账户配置管理（读取JSON，添加/移除账户功能）
│   └── strategy.py        # 跟单策略相关（正向/反向逻辑封装，可扩展）
├── gui/
│   ├── __init__.py
│   ├── main_window.py     # GUI主窗口逻辑，继承自QMainWindow
│   ├── controls.py        # GUI控件定义（如自定义的表格模型等）
│   └── resources.qrc      # GUI资源（图标等）
└── logs/
    └── trade_copier.log   # 日志文件
```

main.py：作为程序启动入口。加载配置文件，创建控制器和GUI对象，实现二者的连接。如采用单进程方案，则可以在main中启动控制器线程，然后启动Qt事件循环；如果GUI和控制器分开进程，也可在main中先启动控制器子进程，再启动GUI进程并建立IPC通信。

TradePublisher.mq5：主账户EA脚本实现。包含 `OnInit` 建立ZMQ PUB套接字 [83] ； `OnTradeTransaction` 事件中调用 `ZmqSend` 发送序列化后的交易JSON [44] 。此外可以实现一个定时器OnTimer来发送心跳信号。这个脚本需在MT5中编译运行，并保持运行状态（可设置EA属性为允许自动交易并在断线重连后自动启动）。

controller/controller.py：控制器核心。初始化ZeroMQ SUB连接 [84] ，并启动一个监听循环协程或线程，不断接收消息 [85] 。每当收到新交易信号，调用相应处理函数： - 将信号转换成统一的内部事件对象，如 `event = TradeEvent(action, symbol, volume, price, sl, tp, ticket)` 。 - 遍历当前活动的副账户列表，针对每个账户根据其策略生成具体 `TradeTask` （比如open任务附带方向、volume等已调整好的参数）。 - 将

任务放入相应账户的执行队列，或者直接调用执行模块（如果每个账户独立线程，则通过线程安全队列发送）。

该模块还负责接收GUI输入，比如GUI请求暂停某账户跟单，可在控制器内将该账户标记为inactive，跳过其任务派发。或者GUI新增账户时，调用account_manager载入新账户并创建对应执行模块。

controller/trade_executor.py：封装与MT5交互的类，如 `TradeExecutor`。每个副账户对应一个TradeExecutor实例，内部持有该账户的MT5连接。当初始化时调用 `mt5.initialize(path)` 并登录账号 [86]。提供方法如 `execute_order(trade_task)`，内部根据任务类型调用不同MT5 API函数完成下单、平仓等操作。包含正向/反向处理和手数调整逻辑（也可挪到strategy模块）：

```
class TradeExecutor:
    def __init__(self, account_config):
        self.cfg = account_config
        mt5.initialize(self.cfg['mt5_path'])
        mt5.login(self.cfg['login'], self.cfg['password'], server=self.cfg['server'])
    def execute_order(self, task):
        # task 包含统一字段: action, symbol, volume, direction, sl, tp, etc.
        # 根据配置决定方向和手数
        volume = (task.volume * self.cfg['lot_multiplier']
                  if self.cfg['lot_mode']=="multiplier" else self.cfg['fixed_lot'])
        direction = task.direction
        if self.cfg['type'] == 'reverse':
            direction = 'BUY' if task.direction=='SELL' else 'SELL'
            # 交换止盈止损
            sl = task.tp; tp = task.sl
        else:
            sl = task.sl; tp = task.tp
        if task.action == 'open':
            request = {
                "action": mt5.TRADE_ACTION_DEAL,
                "symbol": task.symbol,
                "volume": volume,
                "type": mt5.ORDER_TYPE_BUY if direction=='BUY' else mt5.ORDER_TYPE_SELL,
                "price": mt5.symbol_info_tick(task.symbol).ask if direction=='BUY' else
mt5.symbol_info_tick(task.symbol).bid,
                "sl": sl, "tp": tp,
                "deviation": 20, "magic": 1000, "comment": "CopiedTrade"
            }
            result = mt5.order_send(request)
            # handle result, log or retry if needed
        elif task.action == 'close':
            # similar approach to close or order_modify
            ...
```

如上，`TradeExecutor.execute_order` 根据账户配置调整了下单参数，实现了正向/反向和手数倍率 [11] [27]。实际代码需更健壮，包括处理挂单请求（TRADE_ACTION_PENDING等）和错误重试等。TradeExecutor也可以设计成继承 `QThread`，使每个执行器在单独线程运行，从而与控制器主线程并行执行。

controller/account_manager.py：提供加载配置、添加账户、移除账户的功能。`load_config()`读取JSON返回配置对象字典 [87]；`add_account(config)`将新账户写入JSON并在控制器中新建TradeExecutor并订阅（如果需要单独订阅主题）。`remove_account(name)`则停止对应执行器线程、注销MT5连接、更新配置等。

controller/strategy.py：封装策略逻辑，例如定义类或函数`apply_strategy(master_event, follower_config)`，返回针对该副账户的跟单任务列表。内部根据follower_config判断跟单方向、挂单是否复制、止盈止损如何处理等等。将这些规则集中在一处，便于以后扩充更多策略类型（比如区间跟单、不跟某些交易等）。

gui/main_window.py：使用PyQt编写主窗口。可以在Qt Designer设计界面然后加载，或纯代码构建。关键点是：启动时从account_manager获取账户列表，填入表格；连接按钮和菜单的槽，例如`start_button.clicked.connect(controller.start)`，`pause_checkbox.stateChanged.connect(lambda: controller.pause_account(acct_id))`等。通过在控制器中发出PyQt信号，GUI槽函数来更新界面。例如控制器有信号`order_copied(account, order_id, result)`，在GUI中连接到一个函数用于在日志窗口追加一条记录。

日志模块：上面的结构未单列日志模块，可以简单在控制器中用Python内置`logging`配置写入文件log/trade_copier.log，或开发一个`logger.py`集中管理日志（区分普通信息和警告错误）。

通过上述模块划分，开发者可以清晰定位各自职责，**EA模块**负责事件捕获发送，**控制器模块**负责信号路由和决策，**执行模块**负责具体下单操作，**GUI模块**负责用户互动和可视化，**配置模块**提供数据支撑。各模块接口清晰（例如TradeExecutor提供标准的execute_order方法，GUI通过控制器接口操作等），新开发人员可快速理解并修改单独模块而不破坏整体结构 [7]。

8. 结论

以上方案融合了事件驱动架构、ZeroMQ高效通信、主从解耦设计以及GUI可视化控制等多方面优点，构成了一个功能完善且高性能的MT5自动跟单系统。通过在主账户端利用MQL5的`OnTradeTransaction`实时推送交易信号 [73]、在核心控制器中并行复制执行，以及在GUI中友好地呈现和管理，我们实现了**毫秒级**的多账户同步交易和**丰富的策略配置**。技术选型上，Python的灵活性与MT5官方API结合，使开发效率和性能得到兼顾 [68]；ZeroMQ确保了各组件松耦合通信的可靠性和低延迟 [59]；模块化分层设计保障了系统的可扩展和可维护性 [7]。本方案已针对3账户（1主2从，含正反向）的场景优化，并预留了进一步扩展到更多账户及跨服务器部署的空间。希望通过本技术文档的详尽描述，为实际开发实施提供清晰指引，搭建一个稳健高效的自动跟单交易平台。

参考文献： 本文档部分内容参考和整合了相关技术资料 [68] [11] [45] 等。具体链接和出处请参见以上引用标注。

---

[1] [4] [9] [28] [29] [32] [38] [39] [42] [43] [44] [63] [64] [65] [66] [68] [69] [74] [83] [84] [85] [86] MT5自动跟单系统技术文档.markdown
file://file-91VJ3k2uFUSVMT6UsAJpP2

[2] [3] [6] [8] [10] [11] [22] [23] [24] [25] [31] [33] [34] [36] [37] [40] [45] [46] [47] [48] [49] [50] [51] [52] [61] [62] [67] [87] 本地多账户 MT5 自动跟单系统技术方案.pdf
file://file-KWjkZsQQgxgtSCxHjFuEkR

[5] [14] [15] [16] [17] [18] [21] MT5自动跟单系统技术文档_grok.markdown
file://file-Ceu74GqAC1MG1RVLMUfmDt

7 12 13 19 20 30 41 58 59 60 70 71 72 73 75 76 77 78 79 80 81 MT5 本地自动跟单脚本_gemini.pdf

file://file-BSktAdMP9hRnbW3fqoWN12

26 27 35 53 54 55 56 57 新速编程 MT4-MT4跟单EA使用说明.pdf

file://file-DYMeLhJhgm3E8BY4e85p2L

82 MT5自动跟单文档_claude.md

file://file-KYN4sP2QgMhiTg9WEkuNMT